HACKATHON HANDOVER NOTE

Meta PyTorch OpenEnv Hackathon

Scaler School of Technology, Bangalore

India AI Hackathon '26

Compiled: 7 April 2026 | Recipient: Incoming Team Member

Sources: official hackathon dashboard, problem statement, sample scripts, bootcamp video transcripts

1. Event Overview

Event Name	Meta PyTorch OpenEnv Hackathon x SST — India AI Hackathon '26
Organiser	Scaler School of Technology (SST), Bangalore
Partners	Meta (primary sponsor) · Hugging Face (ecosystem) · PyTorch (framework)
Focus Technology	OpenEnv — open-source RL environment framework by Meta & Hugging Face
Expected Participants	70,000+ developers across India
Total Prize Pool	$30,000 USD
Who Should Participate	Developers, ML engineers, CS students in India. No prior RL experience needed.
Prerequisites	Basic Python (3.10–3.12), some ML familiarity, GitHub account, Docker
Support Email	help_openenvhackathon@scaler.com
Discord Community	discord.gg/Dedhy5pkWD (all announcements, mentor access, team matching)
Main Page	scaler.com/school-of-technology/meta-pytorch-hackathon
Dashboard / Submit	scaler.com/school-of-technology/meta-pytorch-hackathon/dashboard

2. Event Timeline

Phase	Date	Details
Registrations Open	14 Mar – 5 Apr 2026	Free. Solo or team (up to 3). Access prep courses immediately.
Round 1 Begins	25 Mar 2026	Problem statements revealed on dashboard. Build begins.
Round 1 Deadline	8 Apr 2026, 11:59 PM IST	Submit HF Spaces URL via dashboard. Team lead submits only.
Round 1 Results	10 Apr 2026	Top ~3,000 teams advance from ~20,000. R2 prep guidelines released.
Grand Finale (Round 2)	25–26 Apr 2026	48-hour in-person sprint at SST campus, Bangalore. Mentors on-site.

⚠ Round 1 submission deadline is 8 April 2026, 11:59 PM IST. Only team leaders can make the final submission.

3. What is OpenEnv?

3.1 The Problem It Solves

When training an LLM with reinforcement learning, developers want to focus on improving the model — not writing plumbing code. Different environments historically have incompatible API specs, which slows down experimentation. OpenEnv solves this by defining a universal interface so that swapping environments requires changing only one line in the training loop.

3.2 Core Concept

OpenEnv is an open-source framework by Meta and Hugging Face for creating standardised, isolated, and reusable environments for training and deploying AI agents. It follows the same concepts as the Gymnasium API (step / reset / state) and packages everything in Docker for reproducibility and scalability.

The ideal training loop looks like:

```
from openenv import CalendarEnv          # swap this one line to change environments
from trl import GRPOTrainer

env = CalendarEnv()                      # everything else stays the same
trainer = GRPOTrainer(model, env, ...)
trainer.train()
```

3.3 The RL Loop (Concepts)

Every OpenEnv environment implements these four primitives — the same ones used in all RL:

- reset() — start a new episode; returns the initial observation
- step(action) — take one action; returns (observation, reward, done, info)
- state() — return the current internal state of the environment
- Reward — a signal over the full trajectory (not just binary end-of-episode). Must provide partial progress.

3.4 How Spaces Are Used (Architecture)

HuggingFace Spaces have three interfaces that OpenEnv exploits simultaneously:

- Server — access the environment via a URL (HTTP API)
- Code repository — clone/push/commit code directly via git
- Docker registry — pull and run the environment as a container locally

Mode	How to Connect	When to Use
Remote HF Space (server)	Pass the HF Space URL to the env class constructor	Quick testing, small batch sizes, no Docker setup needed
pip install from Space (client)	pip install git+<space_url> then import client code	Run client code locally while pointing at remote server

Local Docker container	docker pull <space_tag>, docker run, pass local URL to env	Full local development, large-scale training, offline use

ℹ For most training workloads Meta engineers recommend running locally via Docker. The HF Space is for discovery and light testing. Compute costs for the environment are negligible compared to GPU costs for training.

4. Project Structure & Architecture

4.1 Scaffolding a New Environment

The OpenEnv CLI generates the full project skeleton. Run this once and all boilerplate is created:

```
pip install openenv-core
openenv init <your-env-name>
cd <your-env-name>
```

The scaffold creates a complete, working (echo-only) environment immediately. All file names are pre-populated with the name you chose. You then only need to add your business logic.

4.2 File Structure

Component	Role
models.py	Define typed Pydantic models: Observation, Action, Reward. These form the contract between the agent and the environment.
<env_name>_env.py	The environment class. Implement reset(), step(action), state(). Business logic lives here.
app.py	FastAPI server that wraps the environment and exposes HTTP endpoints. Generated by the CLI scaffold.
Dockerfile	Containerises the FastAPI server. Usually needs no changes unless dependencies are outside Python.
openenv.yaml	Metadata file. Describes the environment, version, tasks. Required for openenv validate to pass.
pyproject.toml	Python project dependencies. Pin openenv-core version here before publishing.
inference.py	Baseline script. Uses the OpenAI client to run a model against the environment. Must be in the repo root.
README.md	Required. Must include: env description, action/observation spaces, task descriptions, setup instructions, baseline scores.

4.3 Where to Write Your Business Logic

There are two primary files to modify — think of them as your two 'theatres of operation':

- models.py — define what Observation, Action, and Reward look like for your specific environment (typed Pydantic models). Example: for a Wordle game, Action = guess (str), Observation = score (list of colours).
- <env_name>_env.py — implement the game/task logic inside reset(), step(), and state(). Example: reset() clears the board; step() drops a piece and checks for a win.

4.4 Key Implementation Notes (from Bootcamp)

- The scaffold can be submitted as-is — it returns zeros but is a valid OpenEnv environment. Build up from there.

- Dockerfile rarely needs modification unless you have non-Python dependencies. Leave it as generated unless needed.
- Pin the openenv-core version in pyproject.toml before publishing. Do not leave it unpinned — a GitHub-linked repo will become unstable over time.
- If your environment uses code outside Python (e.g. a game written in another language), use Docker to manage those dependencies via the Dockerfile.
- CPU cost of running an environment is fractional compared to GPU training cost — do not worry about hosting costs for small-scale use.

---

5. Round 1 — Full Requirements

5.1 The Task

Build a complete, real-world OpenEnv environment that an AI agent can learn from through the standard step() / reset() / state() API.

> **⚠ The environment must simulate a task humans actually do. Not games, not toys. Examples: email triage, code review, data cleaning, scheduling, customer support, content moderation.**

5.2 Functional Requirements

Real-world task simulation

- Must model a genuine task with practical value — not a toy or trivial puzzle
- Examples of valid domains: email triage, code review, data cleaning, scheduling, customer support, content moderation, document summarisation, bug classification

OpenEnv spec compliance — implement all of these:

- Typed Pydantic models: Observation, Action, Reward
- step(action) → returns observation, reward, done, info
- reset() → returns initial observation (clean state)
- state() → returns current internal state
- openenv.yaml with environment metadata
- Passes openenv validate without errors

Minimum 3 tasks with agent graders

- Each task defines a concrete objective an agent must accomplish
- Each task has a programmatic grader that scores performance 0.0 → 1.0
- Tasks must span a difficulty range: easy → medium → hard
- Graders must have clear, deterministic success/failure criteria
- The hard task should genuinely challenge frontier models

Meaningful reward function

- Must provide signal over the full trajectory — not just binary end-of-episode
- Must reward partial progress toward task completion
- Must penalise clearly undesirable behaviour (e.g. infinite loops, destructive actions)

Baseline inference script (inference.py)

- Uses the OpenAI API client to run a model against the environment
- Reads credentials from environment variables (see Section 6)
- Produces a reproducible baseline score on all 3+ tasks
- Must be named exactly inference.py and placed in the repo root

5.3 Non-Functional Requirements

Deployment

- Must deploy to a Hugging Face Space tagged with openenv
- Space URL must return HTTP 200 and respond to reset()
- Must include a working Dockerfile (docker build + docker run must succeed cleanly)

Documentation (README.md — mandatory)

- Environment description and motivation
- Action and observation space definitions
- Task descriptions with expected difficulty level
- Setup and usage instructions
- Baseline scores for all tasks

Infrastructure restrictions

- Inference script must complete in under 20 minutes
- Environment and inference must run on: vCPU = 2, memory = 8 GB

---

6. Mandatory Technical Instructions

6.1 Required Environment Variables

These must be defined in your environment configuration before submitting:

Variable	Purpose
API_BASE_URL	The API endpoint for the LLM (e.g. https://router.huggingface.co/v1)
MODEL_NAME	The model identifier to use for inference (e.g. Qwen/Qwen2.5-72B-Instruct)
HF_TOKEN	Your Hugging Face API key
LOCAL_IMAGE_NAME	Name of the local Docker image if using from_docker_image()

6.2 Inference Script Rules

⚠ Any deviation from these rules will result in incorrect evaluation scoring and potential disqualification.

- File must be named inference.py (exact name, exact location: repo root)
- Must use the OpenAI client for all LLM calls — no other HTTP clients
- Must read API credentials from environment variables (not hardcoded)

6.3 Mandatory stdout Log Format

The script must emit exactly three line types to stdout, in this exact format:

```
[START] task=<task_name> env=<benchmark> model=<model_name>
[STEP]  step=<n> action=<action_str> reward=<0.00> done=<true|false> error=<msg|null>
[END]   success=<true|false> steps=<n> score=<score> rewards=<r1,r2,...,rn>
```

Formatting rules:

- One [START] line at episode begin
- One [STEP] line per step, immediately after env.step() returns
- One [END] line after env.close(), always emitted — even on exception
- reward and rewards formatted to 2 decimal places
- done and success are lowercase booleans: true or false
- error is the raw last_action_error string, or null if none
- All fields on a single line — no newlines within a line
- Each task score must be in [0, 1]

Example output:

```
[START] task=click-test env=miniwob model=Qwen3-VL-30B
[STEP] step=1 action=click('123') reward=0.00 done=false error=null
[STEP] step=2 action=fill('456','text') reward=0.00 done=false error=null
[STEP] step=3 action=click('789') reward=1.00 done=true error=null
[END] success=true steps=3 score=1.00 rewards=0.00,0.00,1.00
```

6b. Practical Setup Tips (from Bootcamp)

6b.1 Critical: Move the Dockerfile to the Repo Root

After running openenv init, the Dockerfile is generated inside the /server subfolder. You MUST move it to the outermost (root) folder of your project before building or submitting. Failing to do this will cause the automated Dockerfile check to fail.

```
# After openenv init <env-name>:
mv <env-name>/server/Dockerfile <env-name>/Dockerfile
```

> ⚠ **Dockerfile must be in the repo root, not inside /server. This is one of the most common submission errors.**

6b.2 Enable the Web Interface (Recommended for Testing)

Add the following environment variable line to your Dockerfile to enable a browser-based UI at /web for manual testing of step, reset, and state:

```
ENV ENABLE_WEB_INTERFACE=true
```

Once running, navigate to http://localhost:<port>/web to interactively send messages and see rewards. This makes it easy to verify your environment is behaving correctly before running inference.

6b.3 UV — Required for Running the Inference Script

UV is a fast Python package manager used to manage the environment's dependencies. It must be installed and synced before executing inference.py. Run these commands inside your project virtual environment:

```
pip install uv
uv sync

# Then run the inference script:
uv run inference.py
```

> ℹ Always use a virtual environment. Create one first with: python -m venv .venv && source .venv/bin/activate

6b.4 Using AI Agents to Build Your Environment

You can use any coding agent (Cursor, Claude, Codex, Open Code, etc.) to help generate your environment. For best results, install the built-in OpenEnv skills into the agent's working directory first — this gives the agent the full API contract and tells it to use the CLI scaffold as a starting point:

```
openenv skills add codex       # for Codex
openenv help                   # see all available skill options
```

Without the skill, agents tend to generate environments that don't conform to the OpenEnv spec. With the skill, they produce spec-compliant code and use the CLI scaffold correctly.

> ℹ You are also encouraged to use AI to update the inference script. Provide your models.py and environment file to the AI and ask it to update inference.py accordingly — it does a good job at this.

6b.5 HF Router — No OpenAI Key Needed

You do NOT need a paid OpenAI API key. The HuggingFace Router provides access to all major LLM providers through a single endpoint using your free HF token:

API_BASE_URL	https://router.huggingface.co/v1
API key	Your HuggingFace token (HF_TOKEN) — NOT an OpenAI key
Model selection	Router auto-selects from models available on HuggingFace. Free credits included.
Recommended model	Qwen/Qwen2.5-72B-Instruct (used in sample script) — or any HF-available model

> ⚠ **Do NOT create an OpenAI API key. Your HF token is sufficient and includes free credits for experimentation and submission validation.**

6b.6 Gradio Custom UI (Recommended for Complex Environments)

For complex environments, adding a custom Gradio interface makes debugging much easier. The default Space comes with a basic UI, but you can extend it:

- HTML component — very generic, supports JavaScript for custom layouts
- Video component — for visual or simulation environments
- 3D support — Gradio supports nearly any UI type including 3D environments

ℹ This is optional but strongly recommended if your environment has non-trivial state that needs to be observed interactively during development.

6b.7 Advanced OpenEnv Features (Optional, Will Score Higher)

The GitHub repo contains additional features beyond the basic spec:

- MCP interface — expose your environment as an MCP server for agent tool use
- LLM-as-judge rubrics — use a language model to judge outputs that cannot be verified programmatically (e.g. writing quality, relevance, helpfulness)
- Process supervision — assign different reward scores to different steps rather than one end-of-episode reward, giving the model a richer learning signal
- Curriculum difficulty — start with easy tasks and dynamically increase difficulty as the agent improves (this pattern is used by frontier labs at scale)

ℹ These features are in the GitHub repo at github.com/meta-pytorch/OpenEnv. Using them will score higher on Environment Design and Creativity criteria.

6b.8 Multiple Submissions Policy

Multiple submissions allowed?	Yes — submit as many times as you want before the deadline
Which is evaluated?	Your most recent submission that passes automated checks
If earlier submissions fail?	No penalty — only your latest passing submission counts
Hard deadline	8 April 2026, 11:59 PM IST — no submissions accepted after this

6b.9 Why the Inference Script Is Mandatory

The inference script is how the hackathon measures your environment's real-world value. An environment is only useful for RL training if a model can actually get a non-zero reward from it. If a model can never produce a correct output — because the task is too hard, the format is wrong, or the reward signal is too sparse — the environment is useless to the training process regardless of how creative the idea is.

The inference script proves that:

- Your environment produces a learnable reward signal
- A model can interact with it and receive meaningful, varying scores
- The grading scheme works correctly and returns values in [0, 1]

⚠ Without a working inference script, your submission cannot be evaluated and will be disqualified. The grader must also NOT return the same score every time — static/constant graders are grounds for disqualification.

---

7. Pre-Submission Checklist

⚠ All five checks must pass or the submission is automatically disqualified.

Check	How It's Verified
HF Space deploys	Automated ping to Space URL — must return HTTP 200 and respond to reset()
OpenEnv spec compliance	Validate openenv.yaml, typed models, step() / reset() / state() endpoints
Dockerfile builds	Automated docker build on submitted repo
Baseline reproduces	Run inference.py — must complete without error and produce scores
3+ tasks with graders	Enumerate tasks, run each grader, verify scores/reward in 0.0–1.0 range

Run the official validation script before submitting:

```
# Install prerequisite
pip install openenv-core

# Download and run the validator
chmod +x validate-submission.sh
./validate-submission.sh <your-hf-space-url> [path/to/repo]
```

The validator performs 3 automated checks in sequence:

1. Pings <space-url>/reset via POST — must return HTTP 200
2. Runs docker build on your repo — must succeed within 600 seconds
3. Runs openenv validate — must pass without errors

ℹ The validator stops at the first failure. Fix each issue before proceeding to the next check.

8. Step-by-Step Submission Guide

8.1 Tools to Install (Before April 1st)

Python 3.10 / 3.11 / 3.12	Required — install one of these versions
Git + GitHub account	Push your submission to GitHub or Hugging Face
Hugging Face CLI	pip install huggingface_hub — needed for deployment
OpenEnv CLI	pip install openenv-core — the main framework
Docker	Containerised testing — essential for the Dockerfile check
Google Colab	Prep course notebooks run here — free tier works
VS Code (recommended)	Best Python + Docker support

8.2 Submission Workflow

4. Log in to the dashboard and choose 1 of the 4–5 problem statements revealed on the platform
5. Scaffold the project: openenv init <env-name> then cd <env-name>
6. IMPORTANT: Move the Dockerfile out of /server to the repo root: mv server/Dockerfile ./Dockerfile
7. Add ENV ENABLE_WEB_INTERFACE=true to your Dockerfile for browser-based testing
8. Write your business logic in models.py and <env_name>_env.py
9. Add your tasks and graders (minimum 3, easy → medium → hard, scores in [0,1])
10. Write inference.py following the mandatory format in Section 6 (or update the sample script using AI)
11. Install UV and sync dependencies: pip install uv then uv sync
12. Test locally: docker build -t <env-name> . then docker run ... and visit /web to verify
13. Run openenv validate to check spec compliance
14. Run the pre-submission validator: ./validate-submission.sh <url> .
15. Run the inference script locally: uv run inference.py — confirm [START]/[STEP]/[END] output
16. Deploy to Hugging Face Spaces: openenv push
17. Paste your HF Spaces URL in the dashboard before the deadline

⚠ Only team leaders can make the final submission. Team members cannot submit independently.

---

9. Evaluation — How Judging Works

9.1 Three-Phase Process

Phase	Type	What Happens

Phase 1: Automated Validation	Pass/Fail Gate	HF Space deploys, OpenEnv spec compliance, Dockerfile builds, baseline reproduces, 3+ tasks with graders
Phase 2: Agentic Evaluation	Scored	Baseline agent re-run, standard Open LLM agent (e.g. Nemotron 3 Super) run against all environments, score variance check
Phase 3: Human Review	Qualitative	Top submissions reviewed by Meta and Hugging Face engineers for real-world utility, creativity, and exploit checks

9.2 Scoring Breakdown

Criterion	Weight	Description
Real-world utility	30%	Does the environment model a genuine task? Would someone use this to train or evaluate agents?
Task & grader quality	25%	Well-defined tasks, clear objectives, graders that accurately measure success, meaningful difficulty progression
Environment design	20%	Clean state management, sensible action/observation spaces, good reward shaping, proper episode boundaries
Code quality & spec compliance	15%	Follows OpenEnv spec, clean project structure, typed models, documented, tested, Dockerfile works
Creativity & novelty	10%	Novel problem domain, interesting mechanics, clever reward design, original approach

9.3 Detailed Scoring Rubric

Real-world utility (30%)

- 0–5: Toy/artificial problem with no practical application
- 6–15: Valid domain but shallow modelling of the real task
- 16–25: Good domain modelling, useful for agent evaluation
- 26–30: Excellent — fills a real gap, immediate value for the RL/agent community

Task & grader quality (25%) — evaluators check:

- 3+ tasks with difficulty range?
- Graders produce scores between 0.0–1.0?
- Graders deterministic and reproducible?
- Hard task genuinely challenges frontier models?

Environment design (20%) — evaluators check:

- reset() produces clean state?
- Action/observation types well-designed and documented?

- Reward function provides useful varying signal (not just sparse)?
- Episode boundaries sensible?

Code quality & spec compliance (15%) — evaluators check:

- openenv validate passes?
- docker build && docker run works?
- HF Space deploys and responds?
- Baseline script runs and reproduces scores?

Creativity & novelty (10%) — evaluators look for:

- Domain not previously seen in OpenEnv?
- Reward design with interesting properties?
- Clever mechanics that make the environment engaging?

9.4 Disqualification Criteria

⚠ **Environment does not deploy or respond**

⚠ **Plagiarised or trivially modified existing environments**

⚠ **Graders that always return the same score (non-deterministic or trivial)**

⚠ **No baseline inference script**

---

10. Grand Finale (Round 2)

Date	25–26 April 2026 (48 continuous hours)
Format	In-person on-campus sprint
Venue	Scaler School of Technology campus, Bangalore
Judging	Meta's global engineering team
Mentorship	Direct on-site access to Meta, PyTorch, and Hugging Face engineers
Expert Talks	Live sessions from Meta, PyTorch, and Hugging Face engineers
Food	All meals covered during the 48-hour finale
Accommodation	Not provided — participants must arrange independently
Merch	Exclusive limited-edition hackathon swag for all finalists
Team formation	Solo qualifiers may opt to team up with other selected builders on campus

11. Prize Structure

Position	Prize (USD)	Additional Benefit
1st Place	$10,000	Interview opportunity at Meta & Hugging Face
2nd Place	$7,000	Interview opportunity at Meta & Hugging Face
3rd Place	$5,000	Interview opportunity at Meta & Hugging Face
4th – 8th Place	$2,000 each (×5 = $10,000)	Code reviewed by Meta engineers
9th – 15th Place	$650 each (×7 = $4,550)	Code reviewed by Meta engineers
All Finalists	—	Exclusive limited-edition hackathon merch + all meals covered

Beyond cash prizes, all finalists receive:

- Code reviewed by Meta engineers shaping agentic AI
- Real open-source contribution — code ships to a Meta-backed project, visible on GitHub
- A credential that carries weight beyond a certificate

12. Team & Eligibility Rules

- Open to: developers, ML engineers, and CS students in India
- Team size: solo or up to 3 members (mixed colleges/companies allowed)
- Only one person (team lead) fills the team form; teammates added by email address
- Each teammate must have their own registered account
- If a teammate has already added you to their team, no action is needed from you

⚠ **Once a team is confirmed it cannot be changed. Solo status is locked for Round 1 only.**

---

13. Preparation Resources

13.1 Official Preparatory Course (4 modules, ~3.5 hours)

Module 1 — Why OpenEnv?	45 min | ESSENTIAL for Round 1
Module 2 — Using Existing Environments	50 min | ESSENTIAL for Round 1
Module 3 — Deploying Environments	45 min | ESSENTIAL for Round 1
Module 4 — Building Your Own Environment	60 min | MOST IMPORTANT for Round 1

ℹ Each module: read the README first, then open the notebook in Google Colab. No local setup needed. Free tier works.

Full course repository: github.com/raun/openenv-course/tree/main

13.2 Tutorial Pages in the OpenEnv Repo

All at: github.com/meta-pytorch/OpenEnv → /tutorial/

- 01-environments.md — environment spec and building guide
- 02-deployment.md — deploying to HF Spaces (the bootcamp focus)
- 03-scaling.md — multi-GPU and concurrent environment scaling
- 04-training.md — end-to-end training with TRL and Unsloth

13.3 Bootcamp Recording

Title	OpenEnv Round 1 Bootcamp: Build Your First RL Environment
Date	Wednesday, 1 April 2026, 8:00 PM IST
Hosts	Ben Burtenshaw (Community Education, Hugging Face) & Pulkit Aneja (Scaler Instructor)
Recording	youtube.com/live/kkCNMz0Ptd8

Content	End-to-end walkthrough: openenv init → build → test locally → push to HF Spaces → deploy → blog post

14. Quick Reference Links

Hackathon main page	scaler.com/school-of-technology/meta-pytorch-hackathon
Dashboard & submission	scaler.com/school-of-technology/meta-pytorch-hackathon/dashboard
OpenEnv GitHub (Meta-PyTorch)	github.com/meta-pytorch/OpenEnv
OpenEnv Course (Hugging Face)	github.com/huggingface/openenv-course
Full prep course repo	github.com/raun/openenv-course/tree/main
Discord (announcements + mentors)	discord.gg/Dedhy5pkWD
Support email	help_openenvhackathon@scaler.com
Bootcamp recording (YouTube)	youtube.com/live/kkCNMz0Ptd8